

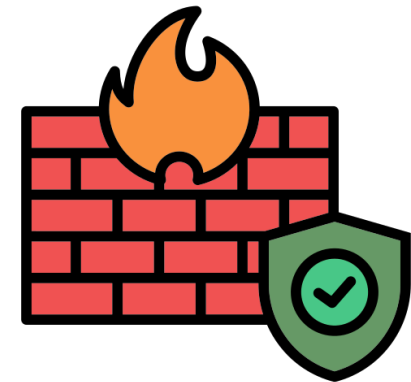
Firewall and Network Segmentation

Cloud Security

CSE4059

Firewalls

- A firewall is a security control that enforces a communication policy on traffic crossing a network or host boundary
- Its decisions may use packet fields, connection state, and application-layer information
- The policy must reflect actual service requirements
 - A correctly enforced but overly broad rule can still permit unwanted communication
- The firewall must lie on the traffic path
 - Traffic that bypasses the firewall is not restricted by its rules



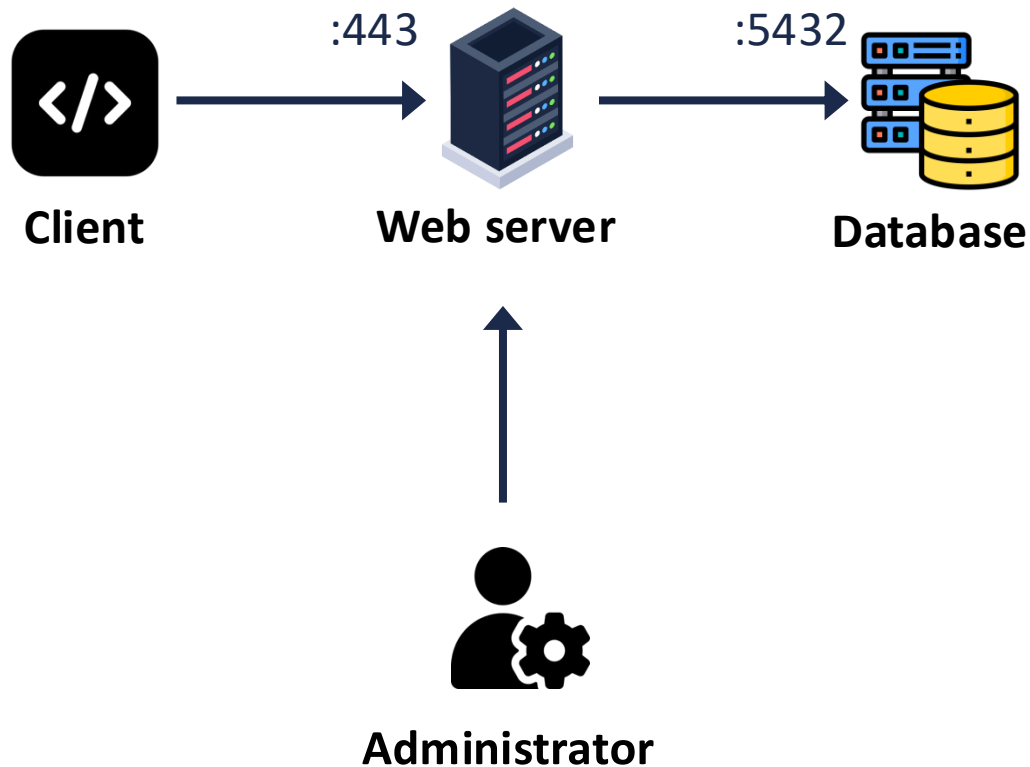
Packet Filtering

- A basic packet filter compares packet-header fields against configured rules
 - Source and destination IP addresses
 - Protocol
 - Source and destination ports
 - Possibly TCP flags
- A rule may combine multiple fields
 - Allowing TCP to one service port is narrower than allowing all traffic to the host
- These fields describe network and transport endpoints
 - Not the application user or the operation requested in the payload

Field	Example
Source	192.0.2.10:53000
Destination	198.51.100.20:443
Protocol / Flags	TCP / SYN

The client uses source port 53000 to send TCP traffic to the web server on destination port 443

Communication Requirements



- Before writing firewall rules, identify the required communication paths
- Clients need access to the web server on TCP port 443
 - They **do not need** direct access to the database
- The web server needs access to the database on TCP port 5432
- Administrators use a separate management path
- Each path represents a specific service requirement, not trust in the entire host

Rules and Default Policy

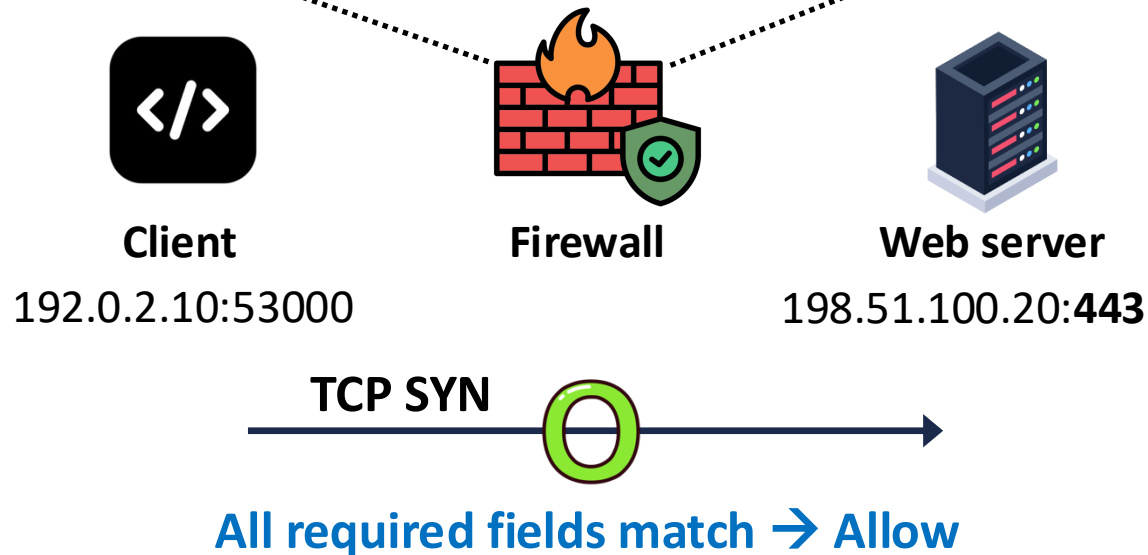
- In this example, rules are checked from top to bottom
 - The first matching rule determines the action

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop

- Rule 1: allows TCP traffic to 198.51.100.20:443
 - All specified conditions must match
- Rule 2: drops all remaining traffic
 - This implements a default-deny policy
- These rules cover only the client-to-server direction
 - We first test an allowed and a denied packet, then examine return traffic

Example: A Permitted Packet

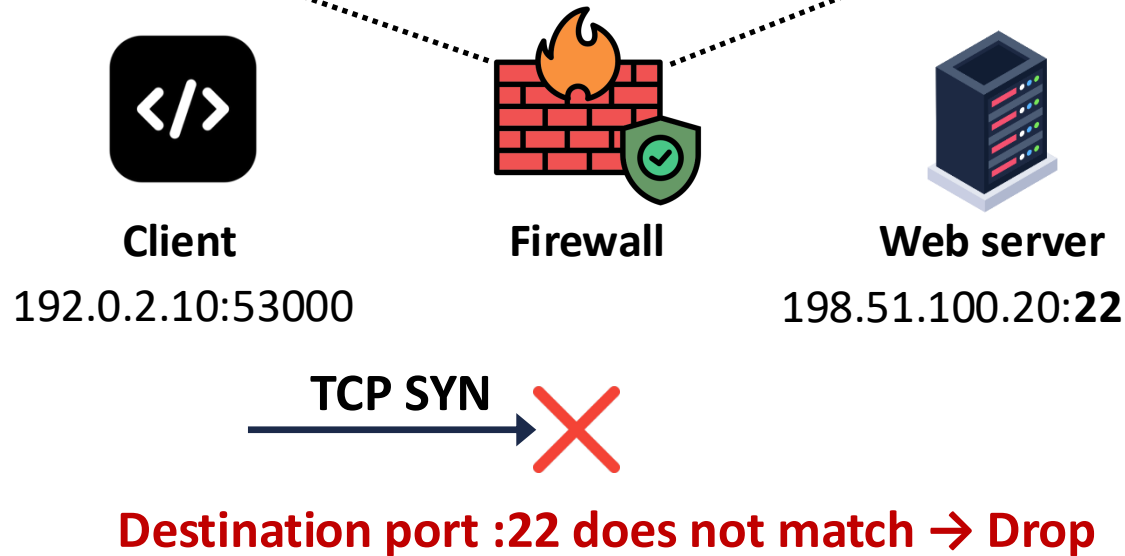
Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow 
2	All remaining packets	Drop



- The client sends a TCP SYN to the web server
- The protocol, destination address, and destination port all match Rule 1
 - The firewall allows the packet
- Port 53000 is the client's source port for this exchange
 - Port 443 is the destination server port

Example: A Denied Packet

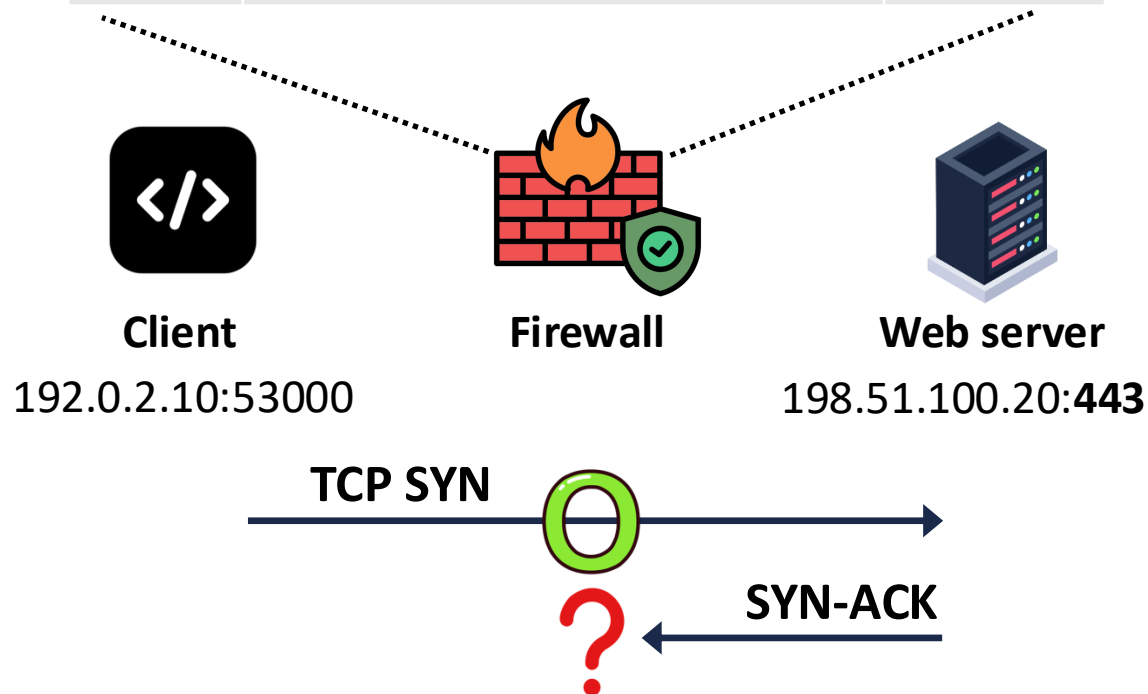
Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop



- Let's change the destination port
 - 443 → 22
- This packet does not match Rule 1
 - The deny rule drops it
- Access to one service does not imply access to every port on the same host
- The web connection on port 443 is still permitted

Request and Reply

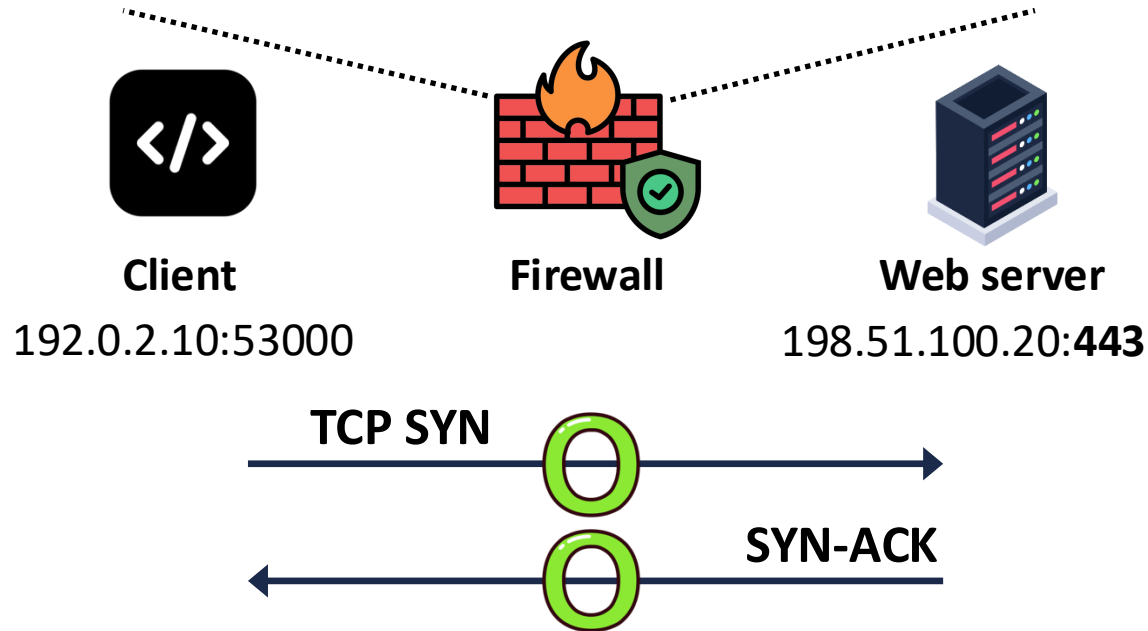
Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop



- The reply reverses the source and destination endpoints
 - Port 443 → 53000
 - port 53000 is an ephemeral client port selected for this connection
- The original rule matching destination port 443 does not match the reply
- The firewall policy must also handle return traffic
 - Can be done with an **explicit** reverse-direction rule
 - or by **remembering** the state of an allowed flow

Stateless Filtering

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	TCP from 198.51.100.20:443 to :53000	Allow
3	All remaining packets	Drop



- A stateless filter evaluates each packet independently against configured rules
- It does not remember previously allowed traffic
 - Allowing (client → server) traffic does not automatically allow the reply
- Return traffic must match a separate reverse-direction rule
- Can packet fields alone tell whether it is really a reply?

Stateless Filtering

- A stateless rule could check the TCP ACK flag to narrow the traffic it allows
 - ex) allow packets from the web server only when ACK is set
- However, a sender can set ACK even when no corresponding prior connection exists

Packet	Matching prior flow?	ACK flag	Rule: ACK set
Legitimate SYN-ACK	Yes	Set	Matches
Unsolicited SYN-ACK	No	Set	Also matches

→ Both a legitimate reply and an unsolicited packet may match the same ACK-based rule

- A stateless filter cannot distinguish them using connection history
 - Stateful filtering

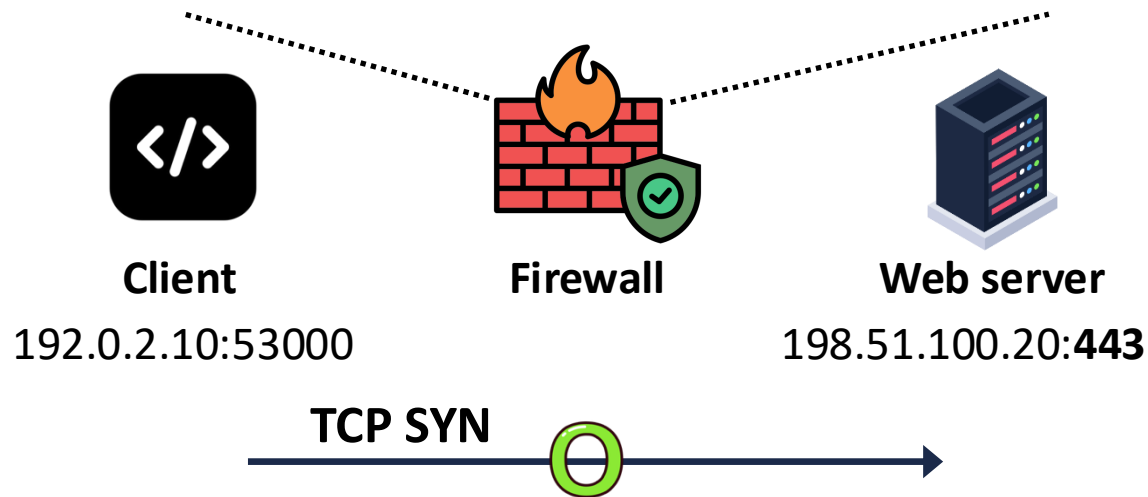
Stateful Filtering

- A stateful filter evaluates packets using both **configured rules** and **remembered connection state**
- For a new connection
 - No matching state entry exists, so the firewall evaluates the new-connection policy
 - If the connection is allowed, the firewall creates a state entry for that flow
- For later packets
 - The firewall checks whether the packet matches an existing state entry
 - Matching return traffic can be recognized using that state, without a broad reverse-direction rule
- Packets that match neither an allowed state nor the applicable policy are dropped

Stateful Filtering: Tracking a New Connection

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop

[State Table] Tracked flow	State
TCP client :53000 ↔ web :443	Pending

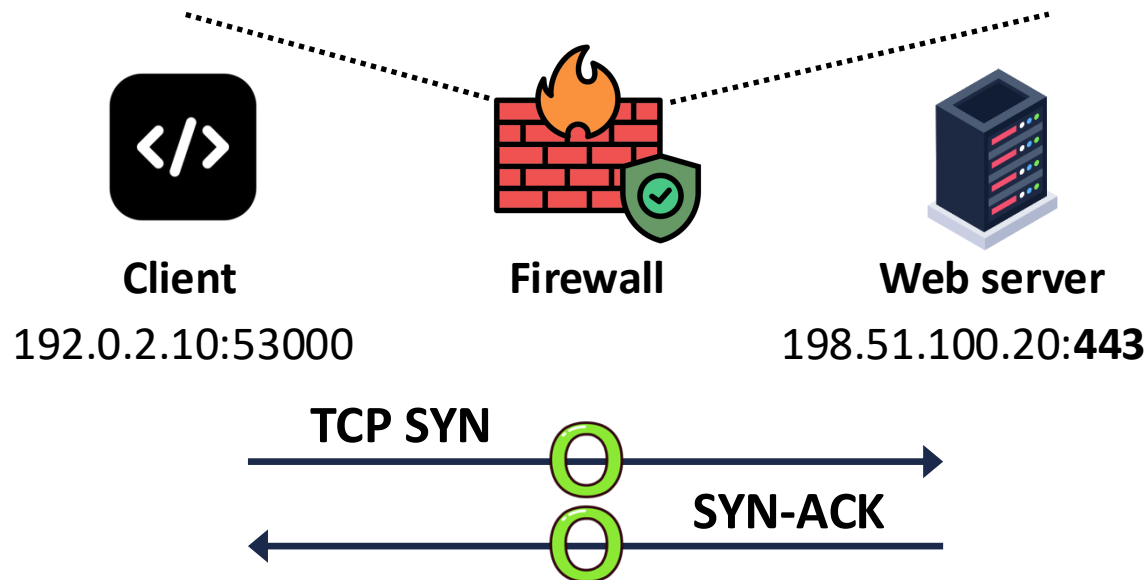


- The same example using the stateful filter
- The SYN matches the firewall policy
 - The firewall allows the packet and creates a state entry for the connection attempt
- The state entry records the flow information and its current state
 - At this point, the connection is pending, not established
- Subsequent packets can be checked against this state entry

Stateful Filtering: Recognizing Return Traffic

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop

[State Table] Tracked flow	State
TCP client :53000 ↔ web :443	Pending

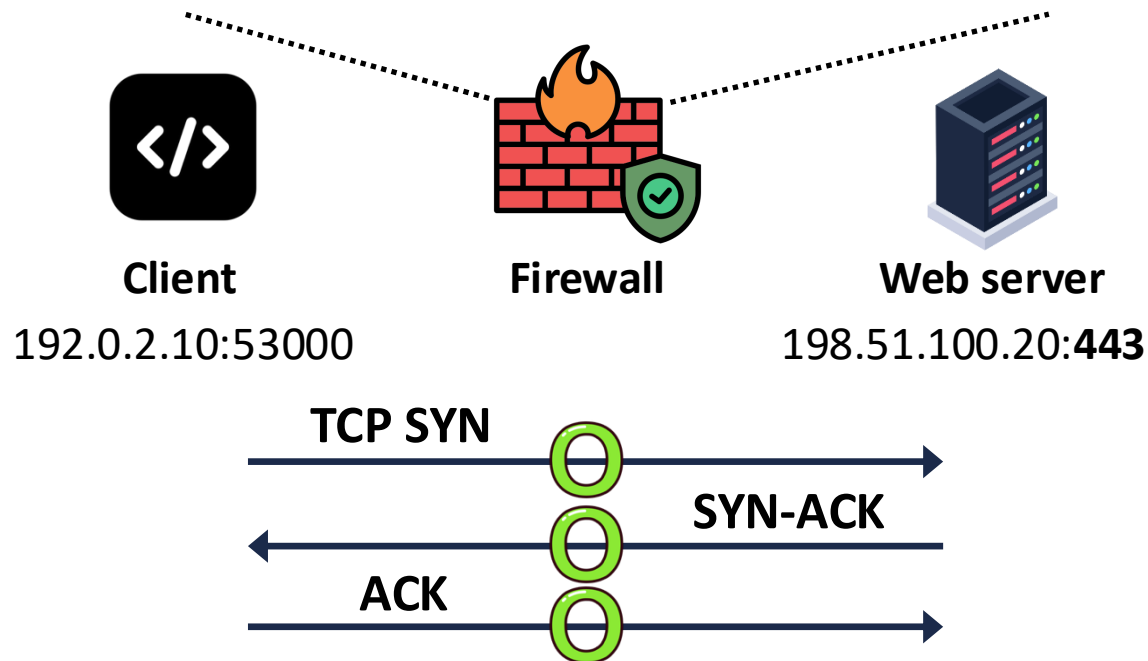


- The web server returns a SYN-ACK
 - 443 → 53000
- The firewall finds the existing pending state entry for this flow
 - The source and destination endpoints are the reverse of the recorded flow
- The firewall recognizes the SYN-ACK as return traffic for the tracked connection attempt

Stateful Filtering: Recognizing Return Traffic

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop

[State Table] Tracked flow	State
TCP client :53000 ↔ web :443	Established

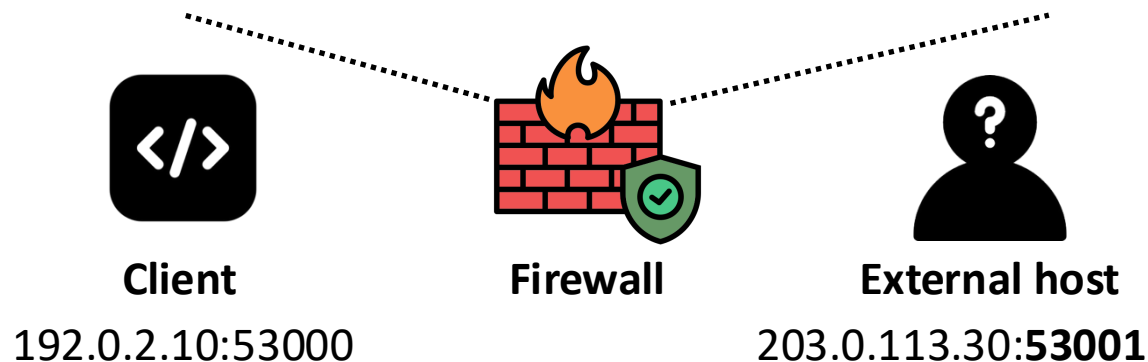


- The web server returns a SYN-ACK
 - 443 → 53000
- The firewall finds the existing pending state entry for this flow
 - The source and destination endpoints are the reverse of the recorded flow
- The firewall recognizes the SYN-ACK as return traffic for the tracked connection attempt
- The client then sends the final ACK
 - The connection state changes (Pending → Established)

Stateful Filtering: Unsolicited Incoming Traffic

Rule #	Condition	Action
1	TCP to 198.51.100.20:443	Allow
2	All remaining packets	Drop

[State Table] Tracked flow	State
TCP client :53000 ↔ web :443	Established



No matching state → evaluate as a new connection
No matching allow rule → Drop

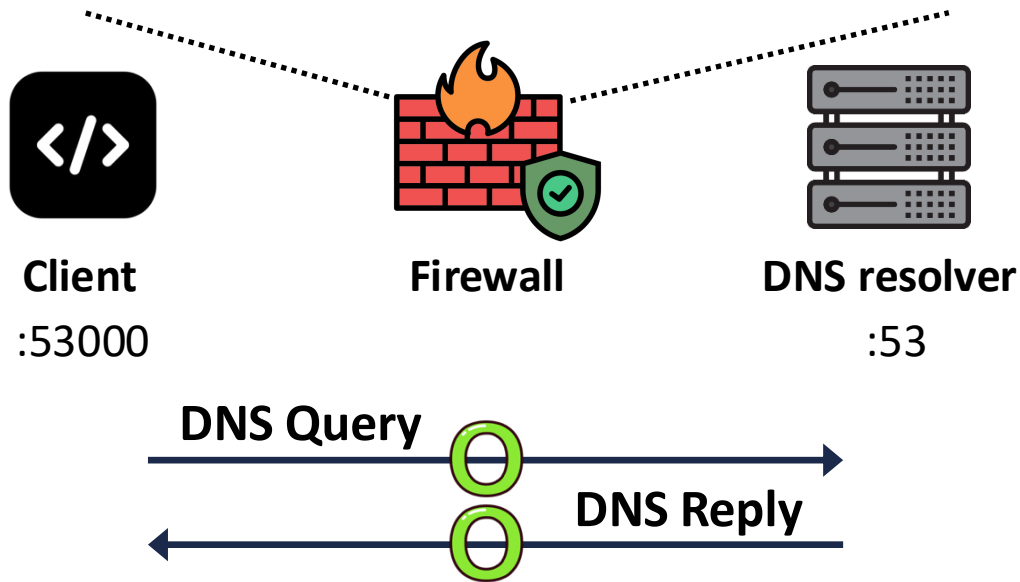
- An external host sends a new TCP SYN
- The packet does not match the established flow in the state table
 - The firewall treats it as a new connection attempt
 - It does not match an allow rule, so the rule 2 drops it
- Stateful return permission applies only to the tracked flow
 - An existing connection does not automatically permit unrelated incoming connections

Stateful Filtering: State Lifetime

- A state entry is temporary
 - It may be removed when the connection closes or after an inactivity timeout
- The exact lifetime depends on the firewall implementation and configuration
- After the state entry is removed, later packets are no longer recognized using that connection state
 - A new connection must satisfy the firewall policy again
- **TCP** provides setup and closure signals (e.g., FIN, RST)
 - Although timeouts are still needed
- **UDP** has no comparable connection signals
 - So flow tracking relies more heavily on temporary tuple-based state and timeouts

Stateful Filtering: UDP Flow Tracking

[State Table] Tracked flow	Lifetime
UDP client :53000 ↔ resolver :53	Until timeout



- UDP has no TCP-style handshake
 - A firewall cannot track connection setup and closure in the same way as TCP
- UDP example: DNS query/reply
 - The client sends a DNS query (:53000 → :53)
 - The firewall can **temporarily** remember the flow information
 - A reply from resolver to client (:53000 → :53) can then be recognized as return traffic
- UDP has no connection-closing handshake
 - The flow entry is typically time-limited
 - The entry is removed after a timeout

Stateless and Stateful Filtering

Aspect	Stateless	Stateful
Decision basis	Packet fields + rules	Packet fields + rules + state
Connection history	Not remembered	Remembered
Return traffic	Separate reverse rule	Matching state entry

- Both still depend on correct rules and placement
 - A firewall cannot control traffic that bypasses it
- Neither determines whether an allowed application operation is authorized
 - That requires application-level controls

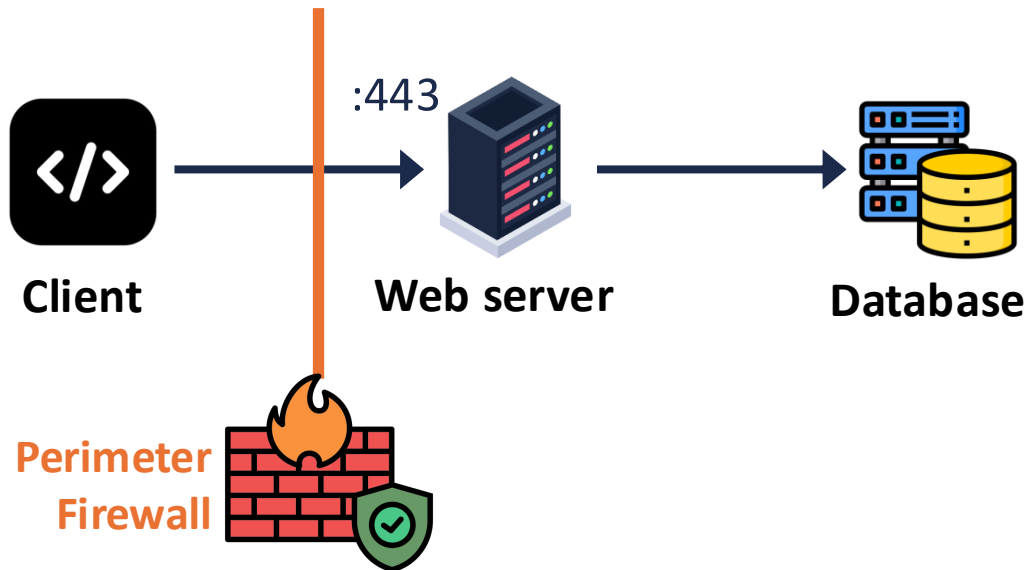
Limits of Packet Filtering

- It cannot enforce application-level authorization
 - Packet filtering based on addresses, ports, and connection state does not determine whether an application operation is authorized
 - The application must authenticate users and authorize their requested operations
 - Network permission to reach a server does not imply permission to perform every operation
- An allowed service may still be vulnerable
 - A server that must remain reachable may contain a software vulnerability
 - If that service is compromised, network segmentation can limit its reach to other systems

Network Segmentation

- Network segmentation divides systems into separate network zones based on their roles and communication requirements
- Communication between zones is restricted by network policies
 - Only required paths and services are allowed
 - For example,
 - Clients may access the public web server
 - The web server may access the database
 - Clients do not need direct access to the database
- Segmentation limits unnecessary reachability
 - If one system is compromised, fewer other systems are directly reachable

A Flat Network

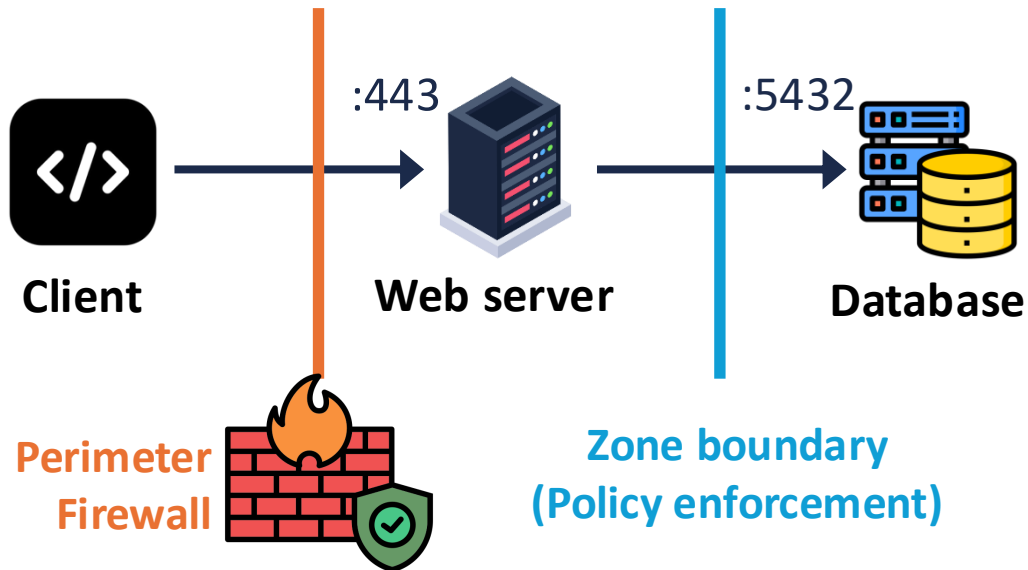


- **Perimeter firewall**: restricts Internet-to-internal traffic
- Internal hosts: can communicate broadly
- Suppose the public web server is compromised
 - The attacker can initiate connections to other internal services from inside the perimeter
- These internal connections do not cross the perimeter firewall
 - Its rules cannot restrict them

Lateral Movement

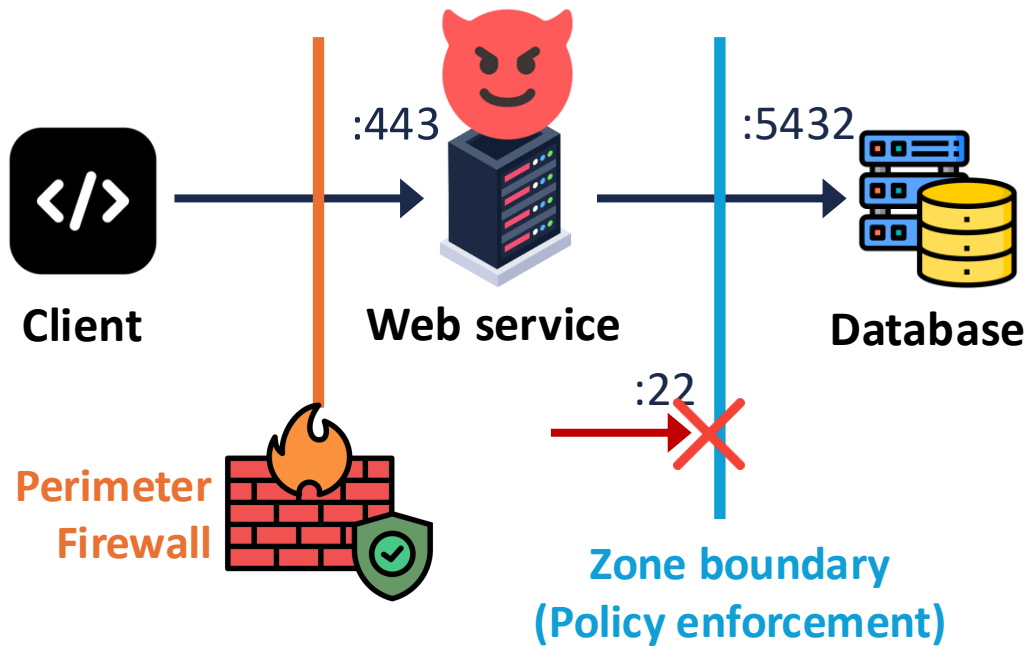
- Lateral movement is the attempt to extend access from a compromised host to other systems in the network
- Suppose the public web server is already compromised
 - The attacker may attempt to access other internal services
- A flat network gives the compromised host more reachable targets
 - Although reachability does not mean successful compromise – credentials, permissions, or vulnerabilities still determine whether further access succeeds
- Network segmentation can remove unnecessary paths for lateral movement

Separating Service Roles



- Place an **enforced zone boundary**
 - between the public web server and the private database
- Allow only the communication required by the application
 - Web → Database: Allow
 - Client → Database: Deny
 - Other new cross-zone connections: Deny
- The application remains functional, while unnecessary internal paths are removed
- Segmentation controls which services are reachable across the boundary

Blocking an Unnecessary Path



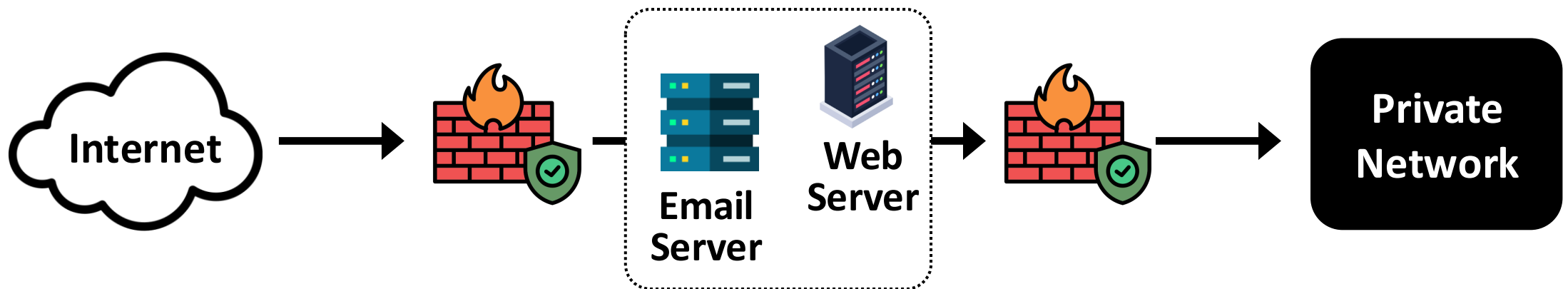
TCP port 5432: allowed by the zone policy

TCP Port 22: denied

- The web server is compromised
 - Attempts a new connection to the database host on TCP port 22
- The zone boundary blocks the connection attempt
 - Even though the attacker is already inside the perimeter
 - Attacker can still use the required path, but cannot freely reach every internal service
- Segmentation removes the **unnecessary path**
 - The required database path on port 5432 remains available

Demilitarized Zone

- Demilitarized Zone (DMZ): A separate network zone for public-facing services
- Public services (ex. web or email servers) are placed between the Internet and the private network
 - (Internet → DMZ) and (DMZ → private network) can be controlled by different policies
- If a DMZ server is compromised, the internal boundary can still restrict its access to the private network
 - Only required internal communication should be allowed



Risk on a Permitted Path

- Even with segmentation, some communication must remain permitted for the application to work
 - ex) The web server still needs access to the database on TCP port 5432
- If the web server is compromised, attacker-controlled code can use the same permitted database path
 - From the network policy's perspective, legitimate and attacker-controlled traffic may use the same endpoints and port
- The database must enforce its own authorization
 - The database account used by the web application should have only the permissions required for its function
- Network segmentation limits which services are reachable
 - Database authorization limits what can be done on a reachable service

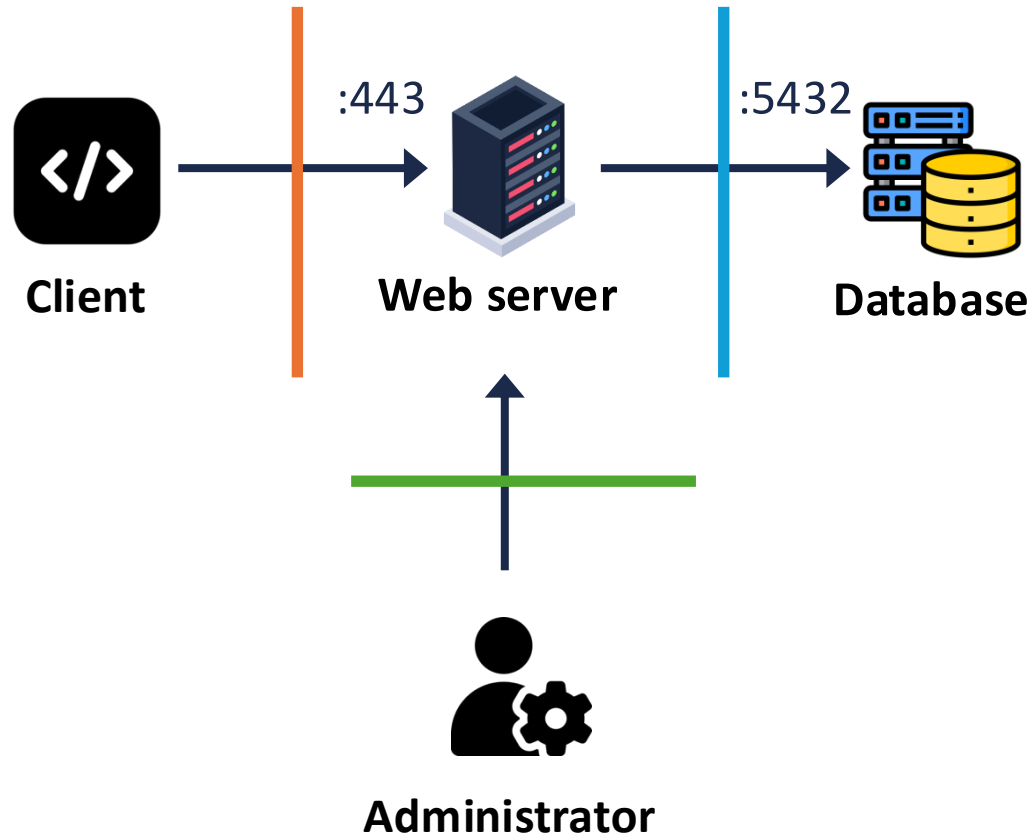
Network and Host Firewalls

- Filtering policies can be enforced at both network boundaries and individual hosts
- A network firewall enforces policy on traffic crossing a shared network boundary
- A host firewall enforces policy on traffic entering or leaving a specific host
- The two provide enforcement at different locations
 - A host firewall can restrict traffic that may not cross an outer network firewall
- Who controls the firewall also matters
 - An attacker with administrative control of a compromised host may be able to modify its local firewall rules
 - A separately managed network firewall can remain outside the compromised host's control

Outbound Policy

- Outbound filtering restricts connections initiated from a protected network or host
 - A compromised system may contact an external controller or send data outside
- Allow only the outbound destinations and services required by the application
 - Blocking all outbound traffic would also break legitimate dependencies
- Outbound filtering still has limits
 - Malicious traffic may use an already permitted path
 - ex) a DNS tunnel may use an approved DNS resolver path
 - Address-and-port filtering alone cannot distinguish every normal DNS query from a data-bearing query

Layered Service Protection



No single control is sufficient on its own

- **Client → Web**

- Network filtering controls whether the web service is reachable
- TLS protects the connection to the intended service

- **Web → Database**

- Network filtering limits the database path to the required service
- Database authorization limits what the application account can do

- **Management**

- Administrative access is restricted to authorized management paths
- Monitoring helps detect suspicious administrative activity and unauthorized configuration changes

Summary

- Packet filtering enforces communication policies using packet and flow information
- Stateless filtering evaluates packets independently, while stateful filtering also remembers permitted flows to recognize return traffic
- Network segmentation limits unnecessary reachability and opportunities for lateral movement
- Protecting a system requires multiple complementary security controls at different layers
 - No single control is sufficient on its own